

ADF 18 most important interview questions

1. What are the different types of Integration runtimes(IR) in ADF, and when should you use each?

Integration runtime is a compute engine that provides computational resources to perform data movement, transformation and orchestration.

3 types of IR:

- Azure IR:** It should be used when we have to perform data movement, transformation and orchestration within the cloud.
- Self Hosted IR:** When any source/destination is an on-prem system, then this IR is required to pull data from the on-prem system.
- Azure SSIS IR:** Specifically built to lift/shift Microsoft SSIS package.

ADF-> Manage tab-> IRs-> AutoResolveIntegrationRuntime will be by default present.

To create a new IR: Click on Add(Choose Self Hosted/ SSIS/ Airflow).

2. What would be the platform for creating Azure Self Hosted IR?

As it requires the compute other than Azure so either we will use a local machine / host a virtual machine.

3. How would you handle schema drift in ADF when dealing with evolving datasets?

Suppose you have a schema defined and known for 4 cols present in source & destination. Now a 5th col started coming in source not present at destination.

As for applying transformations we will use Dataflows.

- When the source dataset is created in mapping dataflows, we get 2 options -> Validate schema & Schema drift. **Schema drift** lets us know that the schema is evolving which is ok. It will just write that 5th col in destination marking its value= NULL for existing records.
- In mapping dataflows we also have an option **Automapping** that eliminates any need to apply schema on the right.
- By using a combination of Schema Drift and Automapping we can handle schema drift for evolving datasets.

4. How would you load multiple data files from REST APIs using SINGLE copy activity?

ADF-> Author tab-> Create a new pipeline(pipAPI)-> Inside the pipAPI

- a. **Drag a ForEach activity.**
- b. Inside parameters tab in ForEach-> Create a parameter(loop_value) of Array type having default value as a list of dict having filenames.
Default value: [{"fname": "x.csv"}, {"fname": "y.csv"}, {"fname": "z.csv"}]
- c. In Settings tab in ForEach-> Item(add dynamic content)-> Choose parameter loop_value i.e. @pipeline().parameter.loop_value
- d. **Inside the ForEach activity drag a copy data activity.**
- e. Go to Source -> Create a source dataset(ds_api)-> Select HTTP as datastore-> Data format as CSV
 - i. Choose the linked service or create LS(ls_api) -> IR: AutoResolveIR-> Base URL: <https://raw.githubusercontent.com/> -> Auth type: Anonymous-> Test the connection-> Hit create
Click on advanced in dataset creation page:
 - ii. Creating parameterized dataset-> Click on Parameters tab-> New->Parameter name: p_filenm of string type with default value = f_name of item of ForEach i.e. @item.fname .
 - iii. Go to the connection tab under advanced dataset only. Click add dynamic content for Relative URL.
[anshikagit/ADF/refs/heads/main/Data/@{choose the parameter from below}](#).
Relative URL finally looks like 
[anshikagit/ADF/refs/heads/main/Data/@{dataset\(\).p_filenm}](#)
- f. Go to Sink-> Create a sink dataset(ds_adls) -> Select datastore as ADLS Gen2 and data format as CSV.
 - i. Choose the linked service or create LS(ls_adls) -> IR: AutoResolveIR-> Storage a/c name: ansadfstrg11->Test the connection-> Hit create
Click on advanced in dataset creation page:
 - ii. Creating parameterized file path for directory and file-> Click on Parameters tab-> New->Parameter name: p_folder & p_file of string type with default value= f_name of item of ForEach along with extension of file(here csv) i.e. @item.fname}.csv

- iii. Go to the connection tab under advanced dataset only. Click add dynamic content in file path for both directory and filename and choose the respective parameter created.
- g. Hit the debug to run the pipeline.

5. How would you design a fault tolerant ADF pipeline to process billions of records daily?

We can leverage below facilities for dealing with above scenario:

- a. **Retry Policy:** Inside an activity we can define the no. of time we want to retry the activity along with the retry interval.
- b. **Parallelism in mapping dataflows:** Ability to execute multiple operations parallelly within a single data flow, or to execute multiple data flows activities in parallel within a pipeline, to improve performance and reduce processing time.
 - i. *Parallelism within a single Mapping Data Flow:* Imagine a data flow that processes customer data and needs to write the results to two different destinations: a SQL database for analytical reporting and a data lake for archival. If these two sinks are branched from the same stream and grouped, enabling "Run in parallel" in the Sink Properties of the Data Flow activity within your pipeline. This instructs Data Flows to write to these connected sinks simultaneously rather than sequentially. This will allow ADF to write to both the SQL database and the data lake concurrently, speeding up the overall data flow execution.
 - ii. *Parallelism across multiple Data Flow activities in a Pipeline:*
 - 1. Consider a scenario where you need to process sales data from one source and inventory data from another, with no interdependencies between the two processing steps. You can create two separate Data Flow activities in your pipeline, each handling one data source. By arranging them in parallel branches, ADF will execute both data flows simultaneously, significantly reducing the total pipeline execution time.

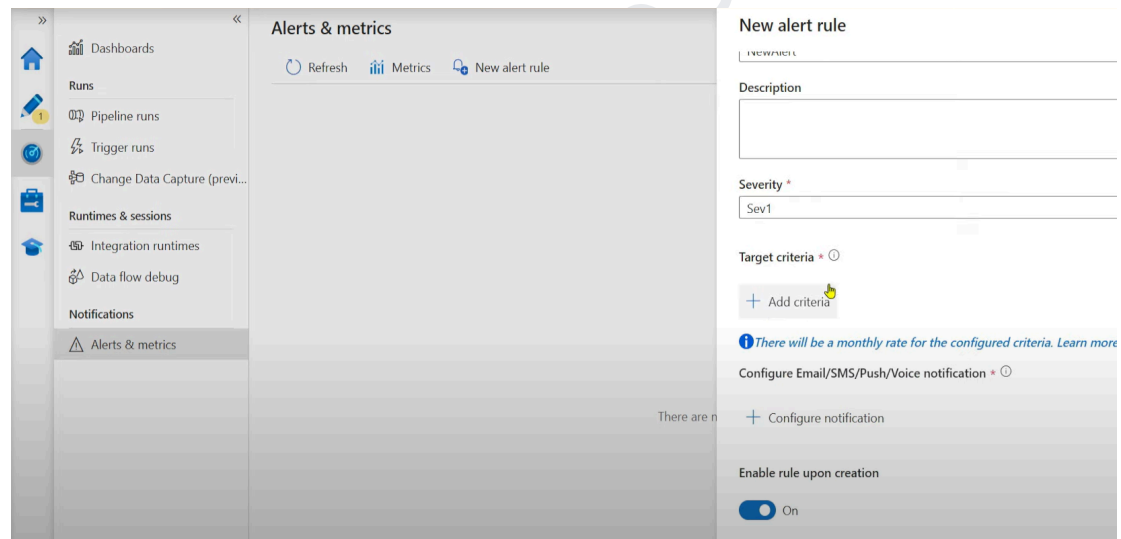
2. If you have a folder containing multiple CSV files, and you want to apply the same data transformation logic to each file using a data flow, you can use a ForEach activity that iterates over the list of files.

By enabling parallel execution within the ForEach, ADF will process multiple CSV files concurrently, rather than one by one, accelerating the overall processing of the folder.

- c. **Failure notification:** We can use create Alert through Monitor tab for a particular activity/pipeline failure.

Process to create an alert is provided below:

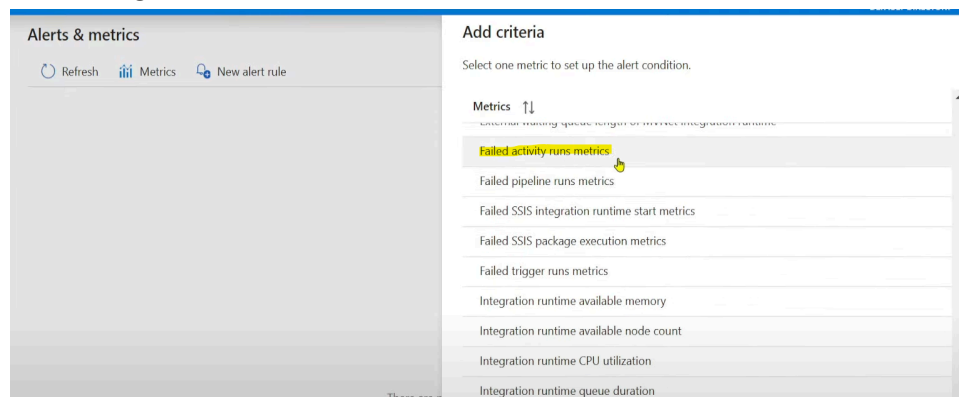
- i. Click the new alert rule -> Provide name(failAlert1)-> Choose severity.



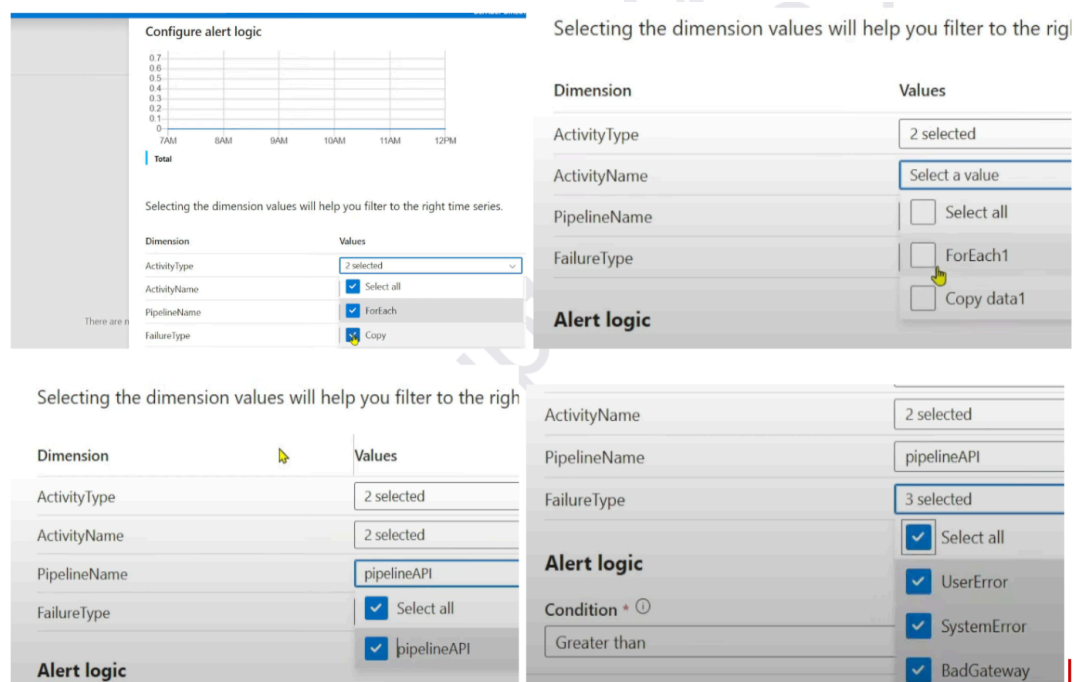
The screenshot shows the 'Alerts & metrics' section of the Azure Data Factory Monitor. The left sidebar contains navigation links: Dashboards, Runs, Pipeline runs, Trigger runs, Change Data Capture (previ..., Runtimes & sessions, Integration runtimes, Data flow debug, Notifications, and Alerts & metrics (selected). The main area is titled 'Alerts & metrics' and includes 'Refresh', 'Metrics', and 'New alert rule' buttons. The 'New alert rule' form on the right includes fields for 'Name' (with 'NEWALERT' as a placeholder), 'Description', and 'Severity' (set to 'Sev1'). It also has a 'Target criteria' section with an 'Add criteria' button and a note: 'There will be a monthly rate for the configured criteria. Learn more'. Below this is a 'Configure Email/SMS/Push/Voice notification' section with a 'Configure notification' button. At the bottom, there is an 'Enable rule upon creation' toggle switch set to 'On'.

- ii. Suppose we want to send out a notification on failure of copy activity made in Ques 4. It means ForEach activity to be included as well because any child activity failure inside a parent activity leads to failure of parent activity.

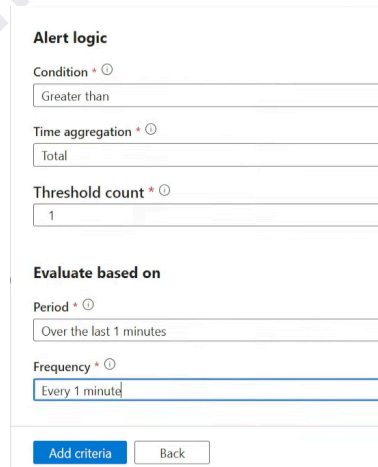
iii. Click target criteria->Add metrics-> Continue



iv. Select activity type(s),name(s),pipelineName(s),FailureType(s)



v. Add the criteria with condition, threshold, period & frequency.



- vi. After adding criteria, click Configure notification-> Add notification-> Select Email/SMS/both-> Provide the details->Hit add notification and you are done with creation of an alert notification.

The screenshot shows the 'Configure notification' tab in the Azure Monitor alert rule configuration. The 'Add notification' panel on the right is open, showing the 'Select which notifications you'd like to receive' section. The 'Email' and 'SMS' notification methods are selected. The 'Email' method is configured with 'email@example.com'. The 'SMS' method is configured with 'Country code' 1 and 'Phone number' 1234567890. The 'Add notification' panel also includes options for 'Azure app push notifications' and 'Voice' notifications.

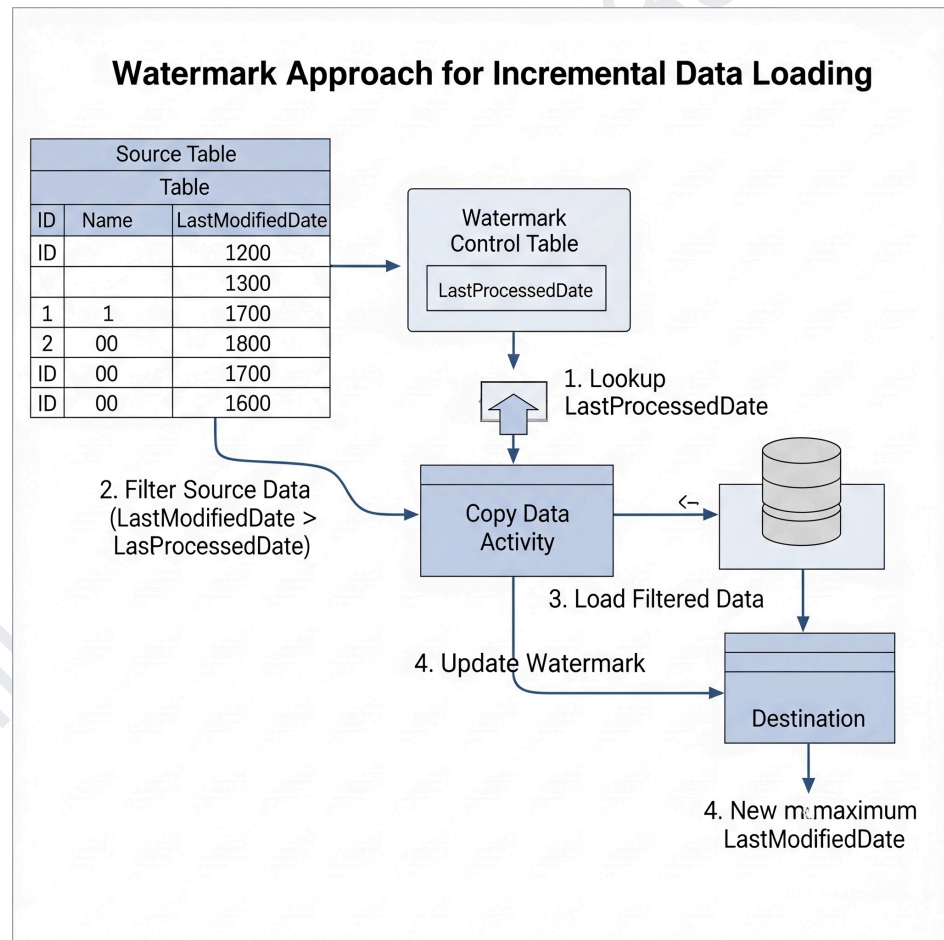
6. Your ADF pipeline is running slow. How would you identify and fix performance bottlenecks?

- a. As a data engineer we will 1st look into the pipeline and see which activity is actually causing the issue. **So, monitoring and finding the root cause will be the 1st step.**
- b. The potential root causes can be:
 - i. If the pipeline is taking a long time while running dataflows, I will try to **prune the data in initial stages.**
 - ii. If the pipeline is taking long in copy activity while pulling data from on-prem to **Azure as self hosted IR's VM(s) is throttling** and is using all the computation it has.

7. How would you implement incremental data load in ADF while working with large transactional DBs?

- a. **Watermark approach:** [Step-to-Step guide](#)
 - i. **Identify a Watermark Column:** Select a column in your source table that changes with each new or updated record, such as a **lastModifiedDate** or an **ID** that always increases.

- ii. **Store the Watermark Value:** Create a separate control table or use a variable in your pipeline to store the watermark value from the previous successful run.
- iii. **Filter Data:** In your pipeline, use a lookup activity to retrieve the last watermark value from the control table. Then, use a copy activity with a dynamic query to select only the records where the watermark column value in the **source** table is greater than the stored watermark value in the **control** table
- iv. **Update the Watermark:** After the copy activity successfully loads the new data, use a stored procedure or another activity to update the watermark value in your control table to the maximum value of the watermark column from the current run.

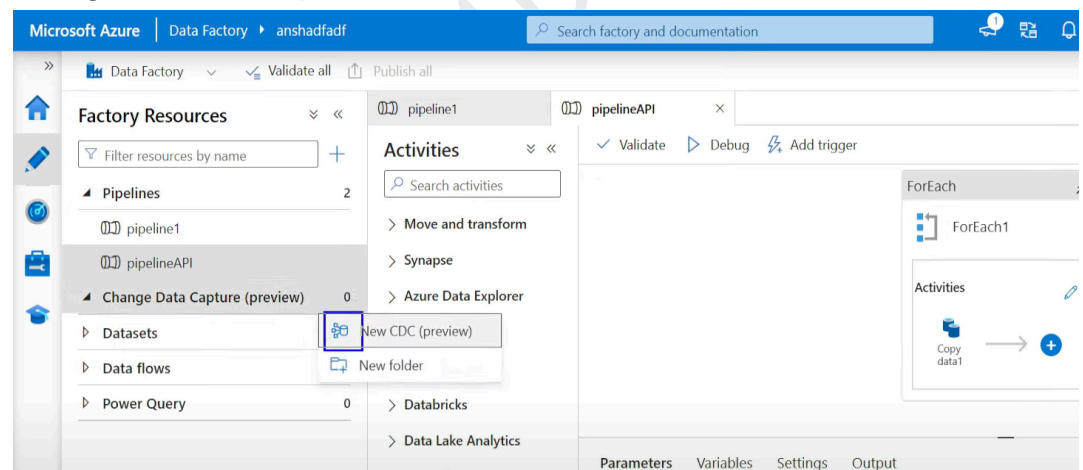


b. CDC(Change Data Capture)

- i. **Enable CDC on the Source:** First, enable Change Data Capture on your source database and the specific tables you

want to track. The database will automatically log all changes to system tables.

- ii. **Use ADF's Built-in CDC:** ADF has features that can directly read from these CDC system tables. This allows you to automatically detect new, updated, and deleted rows.
- iii. **Extract and Apply Changes:** You can use a dedicated CDC source in an ADF data flow or a copy activity that queries the CDC tables. The `__$operation` field in the CDC tables indicates the type of change (e.g., insert, update, delete), which you can use to apply the correct logic to your destination.
- iv. **Manage Checkpoints:** ADF automatically manages the "checkpoint" or last processed position (often a Log Sequence Number or LSN) for each run, ensuring that you only process changes since the previous run.



8. You have a JSON file with an unknown structure. How would you process it in ADF ?

By enabling the Schema Drift it will help in schema evolution.

In dataflows, by turning on debug mode we can preview the schema and we can apply flatten transformation to flatten JSON.

9. How would you implement a real time streaming pipeline in ADF using ADF only?

Suppose a scenario that a file arrives in an ADLS container and you have to perform copy activity on it to another container continuously.

Event Based Trigger can be used here which will automatically trigger the pipeline based on the arrival of the file in the container.

Note: We would be able to use this trigger only after doing the below (earlier it was allowed normally but now not):

Subscriptions->Register Event Trade Services

10. How would you load data to diff locations based on file name in ADF?

- a. **Get File List (Get Metadata Activity):** The pipeline starts with a **Get Metadata** activity. This activity is configured to get the child items (a list of files) from your source folder in a storage account.
- b. **Iterate Over Files (For Each Activity):** The output of the **Get Metadata** activity is passed to a **For Each** activity. This allows the pipeline to loop through each file found in the source folder.
- c. **Conditional Routing (If Condition Activity):** Inside the **For Each** loop, an **If Condition** activity is used to check the file name for a specific pattern or value. This is where you implement the logic to decide the destination. You can use expressions to check if the file name **contains**, **starts with**, or **ends with** a specific string. For example, if the file name contains "sales," it goes to one destination; if it contains "marketing," it goes to another.
- d. **Load Data (Copy Data Activity):** Each branch of the **If Condition** activity contains a **Copy Data** activity. Each **Copy Data** activity is configured with a different sink (destination dataset). The source dataset for all of them would be the current file being processed in the loop, and the sink would be the specific destination you defined.

11. How would you store all the filenames in a variable & no. of files in a folder?

- a. The **Get Metadata activity**, when configured for **Child Items**, returns a single array containing the names of all the files and subfolders in the specified location.
- b. The **For Each activity** is then used to loop through this array. Inside the loop, you can process each item (each filename) individually.
- c. The **Set Variable** and **Append Variable** activities are used to store and manipulate these lists of file names. The video demonstrates using a **Set Variable** activity to assign an entire list of file names to a

pipeline variable. You would typically use the **Append Variable activity** when you're building a list dynamically within a loop, adding one item at a time. Eg. if you were looping through multiple folders and wanted to combine the filenames into a single list, you would use the Append Variable inside the loop for storing all filenames.

- d. **Get the Count from the First Variable:** Use a second **Set Variable** activity to store the count. Create a new variable of type **Integer**, for example, **fileCount**. In its **Value** field, use the following expression to count the number of items in the **fileNamesArray** variable: `@length(variables('fileNamesArray'))`

12. Design a pipeline in ADF for copying only those files from source which aren't available in destination ?

- a. **Get Source File List:** Use the first **Get Metadata** activity to get a list of all files from your source folder.
- b. **Get Destination File List:** Use a second **Get Metadata** activity to get a list of all files from your destination folder. Store this list in a pipeline variable.
- c. **Iterate and Compare:** Use a **For Each** activity to loop through the array of files from the **source Get Metadata** activity's output.
- d. **Implement the Condition:** Inside the loop, use an **If Condition** activity to check if the current file name from the source exists in the destination file list that you stored in the variable. The expression for this condition would be something like:
`@not(contains(string(variables('destinationFiles')), item().name))`
- e. **Copy the File:** Place a **Copy Data** activity inside the **True** block of the **If Condition**. This ensures that the copy operation only runs if the file is **not** found in the destination.

13. How would you call the secrets stored in the Key Vault in ADF ?

Creation of secret in Key Vault:

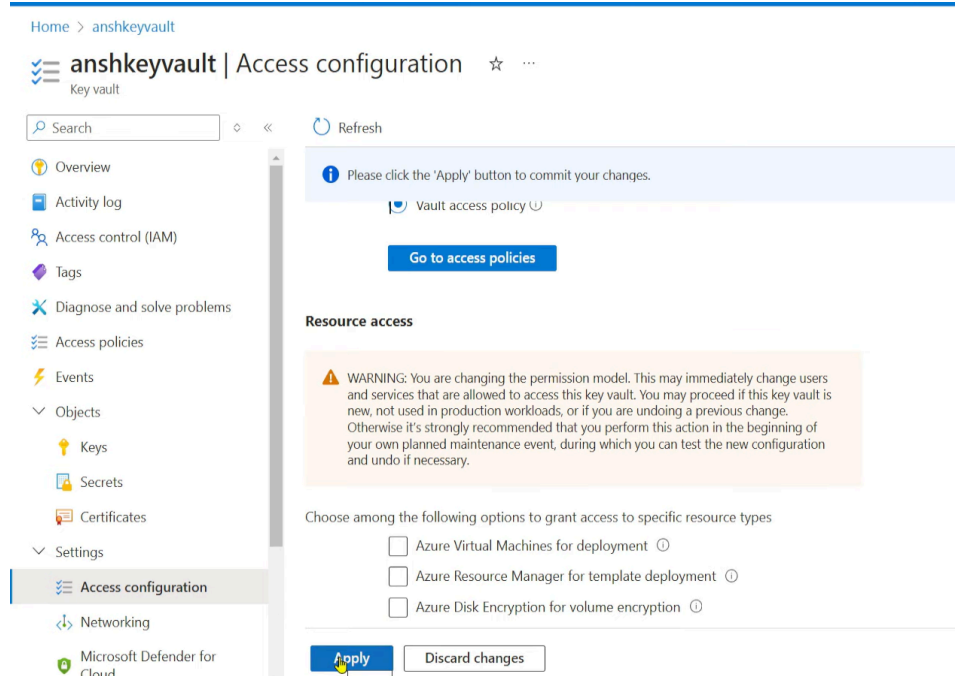
- In the Azure Portal, search for "Key Vaults" and select Create. Fill in the basic details like subscription, resource group, name, and region. The name must be globally unique.
- Go to its IAM -> Assign **Key Vault Administrator** role to the ID which will authorise you to create secrets in the key vault.
- Go to secrets inside the key vault and create one.

The screenshot shows the 'Create a secret' form in the Azure Portal. The breadcrumb navigation at the top reads 'Home > anshkeyvault | Secrets >'. Below the title 'Create a secret' is a plus icon. The form contains the following fields and options:

- Upload options:** A dropdown menu set to 'Manual'.
- Name ***: A text input field containing 'adfdemosecret'.
- Secret value ***: A text input field with masked characters (dots).
- Content type (optional)**: An empty text input field.
- Set activation date**: A checked checkbox.
- Activation date**: A date picker showing '02/02/2025' and a time picker showing '6:15:54 pm'. Below these is a dropdown menu for the time zone, currently set to '(UTC-04:00) Atlantic Time (Canada)'.
- Set expiration date**: An unchecked checkbox.
- Enabled**: Radio buttons for 'Yes' (selected) and 'No'.
- Tags**: A text input field showing '0 tags'.

At the bottom of the form are 'Create' and 'Cancel' buttons.

- Providing 3rd party access of Key Vault(FYI not used in this scenario):** To provide access of the secrets to **any 3rd party** like Databricks: Key vault -> Access configuration -> Choose **Vault access policy to grant data plane access**(by default we have **Azure RBAC** and used in given scenario). We can also configure the access during creation of key vault resource instead of the above hustle.



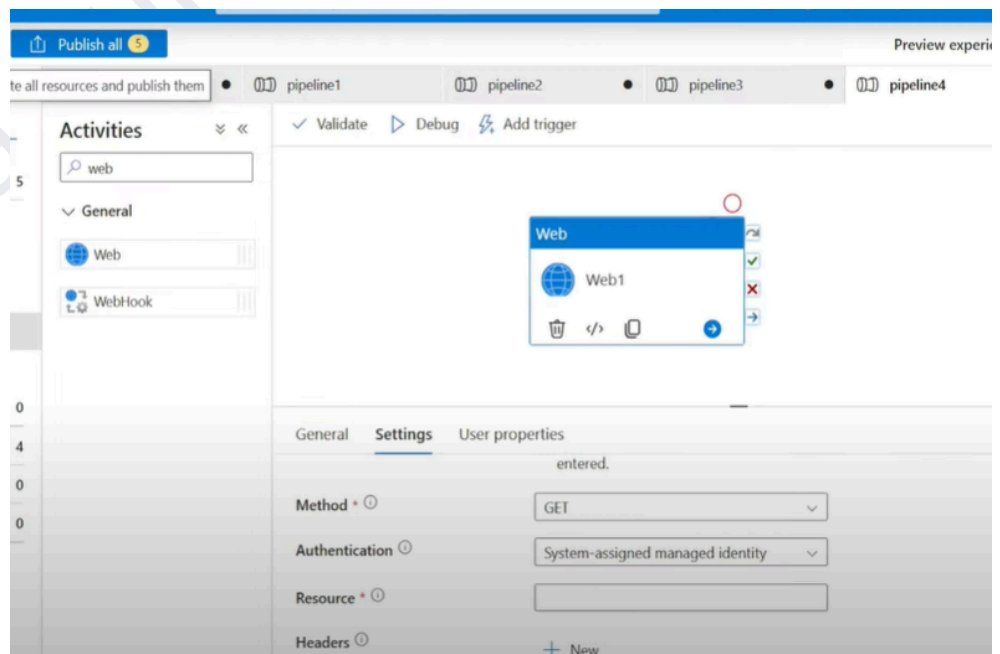
- e. **Set Access Policies for ADF:** This is a critical step. Once the Key Vault is created, navigate to **Access Policies** or **Access Control (IAM)**. You must grant your Azure Data Factory permissions to read secrets from this Key Vault. For standard scenarios, grant the **Get** and **List** permissions for secrets to your Data Factory's Managed Identity.

Calling the secret in ADF:

- a. **The Standard Method: Using a Linked Service** is the most common and recommended approach for securely managing connection strings.
 - **Create a Key Vault Linked Service:** In your ADF instance, go to the **Manage** tab, then **Linked services**, and create a new linked service for **Azure Key Vault**. Point it to the Key Vault you created.
 - **Reference the Secret in Another Linked Service:** When creating a linked service for a data store (e.g., SQL Database), you will see an option to reference a secret for sensitive fields like the password. Choose the Key Vault Linked Service you just created and provide the name of the secret (e.g., **MyDatabasePassword**).

2. Advanced Method: Using the Web Activity is used when you need to retrieve a secret dynamically during a pipeline run, often for use in an activity that doesn't have native Key Vault integration (like a Web Activity for an API call).

- **Use the Web Activity:** Add a **Web** activity to your pipeline. Configure its settings to make a REST API call to your Key Vault to get the secret's value. The URL will be **(Secret Identifier)?api-version=7.4** copy it: `https://<your_key_vault_name>.vault.azure.net/secrets/<your_secret_name>?api-version=7.4`.
- **Configure Authentication:** Set the **Authentication** to **Managed Identity** and the **Resource** to <https://vault.azure.net>.
- **Configure Method:** Get
- **Use the Output:** The secret's value will be in the output of the Web Activity. You can reference it in a subsequent activity using an expression like `@activity('Web1').output.value`.



14. Your pipeline processes large datasets & sometimes runs OOM. How would you avoid this in ADF ?

- a. **Optimize Your Integration Runtime (IR):** The IR is the compute engine for your pipeline. The right configuration can prevent OOM errors by providing enough memory for your workload.
 - **Mapping Data Flows:** If an OOM error occurs, you can scale up the Azure IR used by your data flow. In the settings, you can increase the Core Count and change the Compute Type to Memory Optimized. This provides the underlying Spark cluster with more resources to process large-scale transformations.
 - **Self-Hosted IR:** If you're processing data on-premises, you can add more nodes to your self-hosted IR to distribute the workload and prevent a single machine from running out of memory.
- b. **Partition Large Datasets:** Breaking down your data into smaller, more manageable pieces is a fundamental strategy for avoiding OOM errors.
 - **Files:** For a folder with many files, use the Get Metadata activity to retrieve a list of all files. Then, use a For Each loop to process each file individually with a Copy Data or Mapping Data Flow activity. This ensures you're never trying to process the entire directory at once.

In the source options of a data flow, you can set the partitioning to single, multiple, or no partition. By enabling partitioning, you can distribute your data and allow it to run in parallel, which helps to solve out-of-memory issues.

- **Database Tables:** When copying from a large database table, you can implement an iterative range copy. Use a Lookup activity to find the minimum and maximum values of a key column (e.g., an ID or a date). Then, use a For Each loop to copy data in smaller, defined chunks (e.g., 100,000 rows at a time) using a parameterized query.
- c. **Tune Activity-Specific Settings:** Certain activities have built-in optimizations to help with memory management.

- **Mapping Data Flow:** If you have a join or lookup transformation, be careful with the Broadcast setting. Broadcasting is great for performance, but if the dataset being broadcasted is too large to fit in memory on all worker nodes, it will cause an OOM error. Consider turning off broadcasting for large streams.

The second option is to prune or filter your data as much as possible in the initial stages of your data flow.

- **Copy Data Activity:** For very large single files, like an enormous XML, JSON, or Excel file, you can run into OOM issues. The Copy Data activity generally handles this well, but for extremely large files, consider using the Binary Copy feature to move the file without reading its contents into memory.

15. Your pipeline runs daily , but sometimes the same records get processed twice. How would you ensure idempotency?

Idempotency means that running the same process with the same inputs twice (or more) will have the exact same effect as running it just once. This indirectly relates to exactly once not multiple times in our case.

UPSERT on file data helps to achieve idempotency which can be enabled using inline datasets(delta lake datasets) in mapping dataflows.

- a. **Source Transformations:** You will need at least two source transformations. The first source will be your **incoming data**, and the second will be your **destination data**. Both can use inline datasets, which means you define the connection details and schema directly in the data flow without a pre-existing dataset object.
- b. **Exists Transformation:** After your two sources are configured, use an **Exists** transformation. This is the core of the idempotency logic. It acts like a **WHERE EXISTS** clause in SQL, checking if rows from your incoming data stream (the left stream) exist in the destination stream (the right stream) based on a key (e.g., a primary key).

- c. **Conditional Split:** Chain a **Conditional Split** transformation after the **Exists** transformation. This allows you to create two separate streams of data based on a condition:

New Records: Use an expression that identifies rows where the key **does not** exist in the destination. These are the records to be inserted.

Existing Records: Use an expression that identifies rows where the key **does** exist. These are the records to be updated.

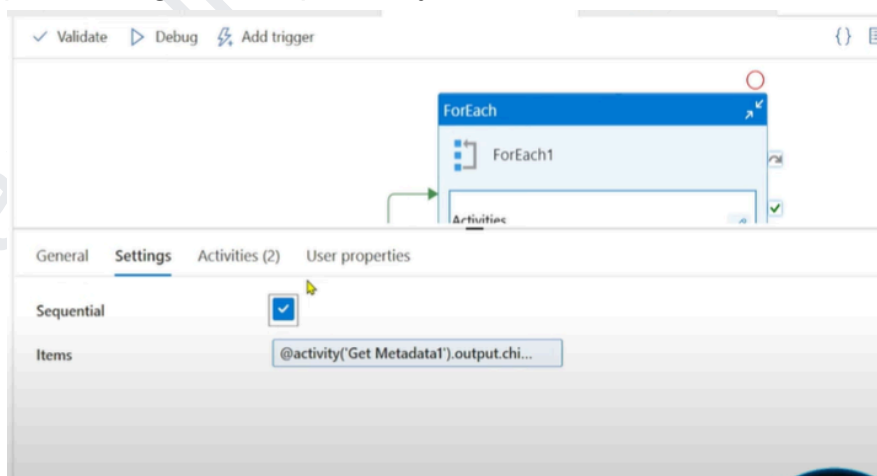
- d. **Alter Row & Sink:** Connect an **Alter Row** transformation to each output stream from the conditional split.

For the "New Records" stream, use an **Alter Row** policy of **Insert**.

For the "Existing Records" stream, use a policy of **Upsert** or **Update**. Finally, connect both **Alter Row** transformations to a single **Sink** transformation, which will write the data to your destination. Be sure to configure the sink's settings to allow the **Insert** and **Upsert/Update** policies and define the key columns.

16. Your ADF pipeline must process files in specific order(e.g. 1stly sales.csv 2ndly product.csv and lastly cust.csv). How would you enforce this in ADF?

By leveraging the **Sequential** option in **ForEach** activity we can enforce processing files sequentially.



17. You have 2 diff files having diff schemas. How would you process both the files using single copy data activity(with mapping)?

- a. **Source Dataset:** Create a source dataset that takes the file name as a parameter. In the file path setting, you will use a dynamic expression like `@item().name`. This expression will pass the name of the current file in the **For Each** loop to the **Copy Data** activity.
- b. **Dynamic Mapping:** Suppose you have clicked import schemas then from code of copy activity for each source file. Run debug and copy the Translator part from JSON.

Delete the Static Mapping: Go to the **Mapping** tab of your **Copy Data** activity. You'll see a list of source and sink columns. To remove the static mapping, you can either delete the entire table or remove each column manually.

Enable Schema Drift: After deleting the schema, ensure that the "Schema drift" option is enabled in the source settings of your **Copy Data** activity. This is the feature that allows the activity to dynamically handle different schemas on the fly.

Map Dynamically: With schema drift enabled, you have two options for mapping:

- **Auto-Mapping:** Simply leave the mapping tab completely blank. ADF will automatically map columns from the source to the sink based on matching names.
- **Explicit Dynamic Mapping:** If you need to transform or rename columns, you can add dynamic mappings using expressions. However, for a simple copy, leaving the mapping blank is the most efficient and straightforward approach.